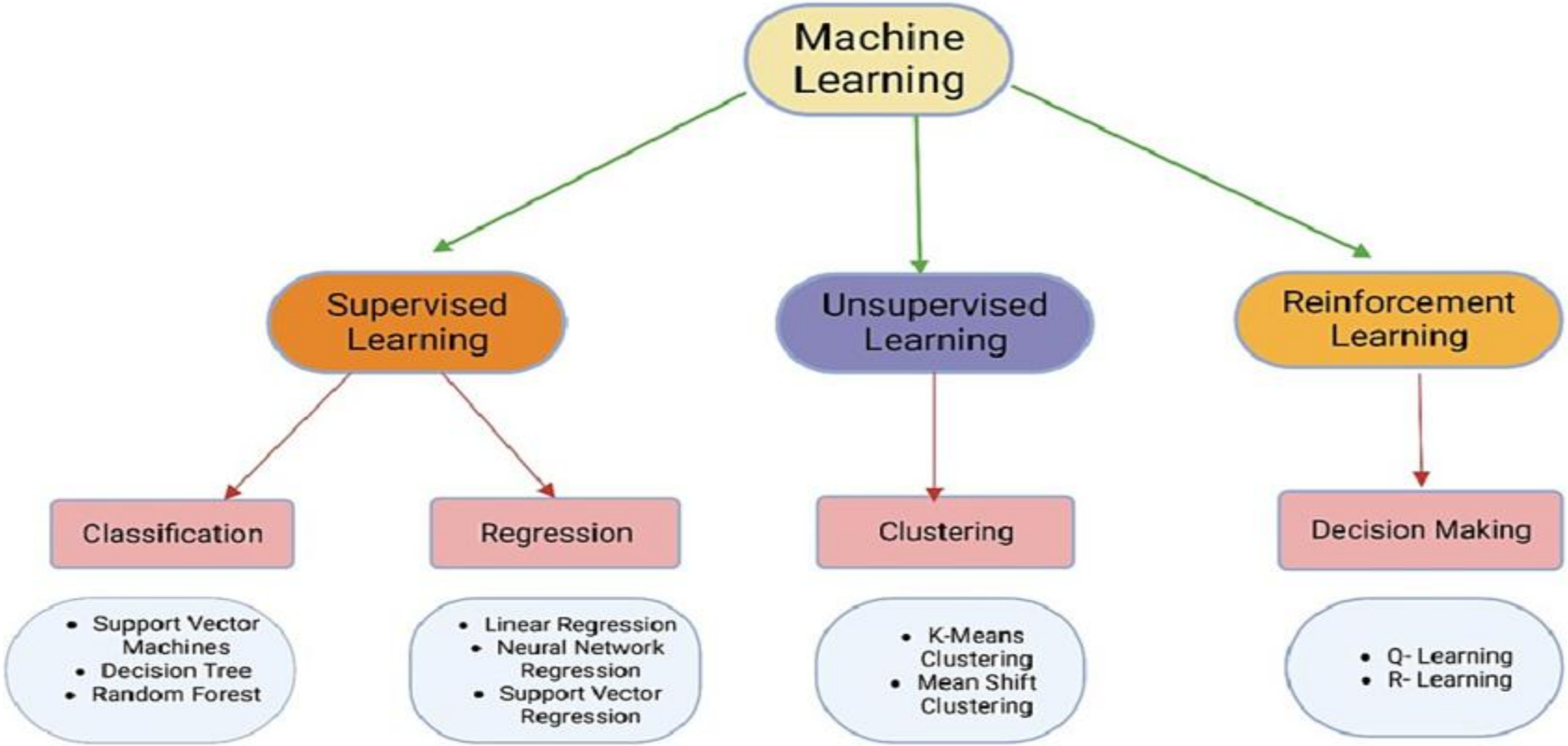
CPE312 Introduction to Machine Learning

Linear regression

Data Preprocessing
Feature Engineering
Model Generalization
Linear Regression Optimization

Friday, 27 March 2026

INTRODUCTION



INTRODUCTION

The performance of models in Machine Learning depends not only on selecting an appropriate algorithm but also on **how the data is prepared and processed before training**.

A typical machine learning workflow includes several stages:

1. Data collection
2. Data preprocessing
3. Feature engineering
4. Model training
5. Model evaluation

Machine Learning

1.Data preprocessing

Feature Scaling

Why Feature Scaling is Needed

Annual Income	0 → 100,000
Age	18 → 60
Pixel Values	0 → 255

Distance Bias

KNN & SVM biased toward large-scale features

Slow Convergence

Gradient Descent converges slowly

Unstable

Numerical instability in optimization

Feature Domination

Some features unfairly dominate others

Solution

Apply Feature Scaling to normalize magnitudes

Models Requiring Scaling

- Linear Regression
- Neural Networks
- KNN / SVM

Feature Scaling

Idea: Make sure features are on a similar scale.

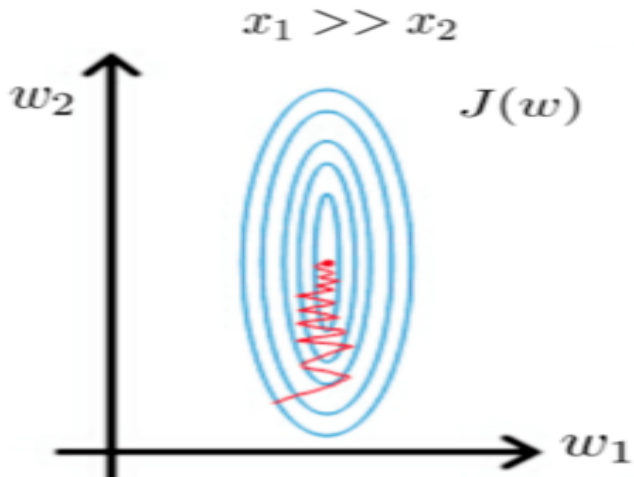
E.g. x_1 = size (0-2000 feet^2)

x_2 = number of bedrooms (1-5)

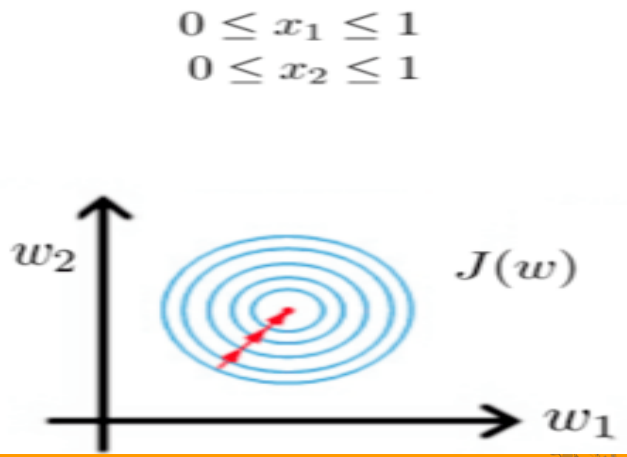
$$x_1 = \frac{\text{size (feet}^2)}{2000}$$

$$\frac{\text{number of bedrooms}}{}$$

Gradient descent without scaling



Gradient descent after scaling variables



Feature Scaling

Get every feature into approximately a range or 0 $\leq x_i \leq 1$

$-1 \leq x_i \leq 1$

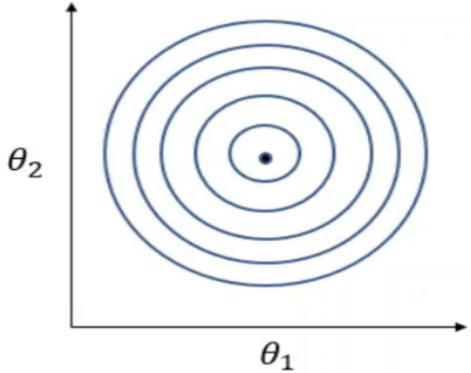
Speed (KM)	Height (M)
200	2.20
160	1.90
180	2.10
240	2.00

Feature scaling

$0 \leq x \leq 1$

Speed (KM)	Height (M)
0.5	1
0	0
0.25	0.66
1	0.33

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$



Standardization (Z-Score Normalization)

$$X_{scaled} = (X - \mu) / \sigma$$

μ (mu)	σ (sigma)
Mean	Standard deviation

- Mean = 0
  Centered around zero
- Std = 1
  Unit variance
- No fixed range
  Unbounded values
- Preserves shape
  Distribution unchanged

Logistic Regression	Linear Regression	SVM
Neural Networks	PCA	KNN

- Annual income (high variance)
- Age
- Debt ratio
- Loan count

Why Standardize?

- Comparable coefficients
- Faster convergence
- Stable optimization

Min-Max Scaling (Normalization)

$$X_{scaled} = \frac{(X - X_{min})}{(X_{max} - X_{min})}$$

Range: [0, 1]

- Fixed Range
  Constrained to [0,1]
- Preserves Shape
  Original distribution
- Outlier Sensitive
  Extremes affect range

- Neural Networks
- Deep Learning
- Image Classification

Pixel values range: 0 → 255

Common preprocessing:

$$X = X / 255$$

- Faster convergence
- Gradient stability
- Numerical precision

Unit Vector Normalization (L1/L2 Norm)

Scales each sample so its norm equals 1

$$X_{scaled} = X / ||X||$$

where $||X|| = \sqrt{(x_1^2 + x_2^2 + \ldots)}$

Direction Focus

Focuses on direction, not magnitude

Similarity-Based

Used in similarity-based systems

NLP

Search Engines

Recommendation Systems

- Bag of Words
- TF-IDF vectors
- Word embeddings

Cosine Similarity:

$$\cos(\theta) = (A \cdot B) / (||A||\ ||B||)$$

Vectors normalized to unit length for comparison

Feature Scaling (Examples)

Features Scaling Methods

1. Min-Max Scaling

Rescaling the values into a range [0, 1]

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Size	Room#
90	3
120	5
150	4
200	6

Size	Room#
0	0
0.27	0.66
0.54	0.33
1	1

2. Standardization (z-score)

Rescaling the values to have a mean of 0 and a standard deviation of 1.

$$x' = \frac{x - \mu}{\sigma}$$

$$x' = \frac{x - \mu}{\sigma}$$

$\mu = 140$	$\mu = 4.5$
$\sigma = 40.62$	$\sigma = 1.118$

Size	Room#
-1.23	-1.34
-0.49	0.44
.24	-0.44
1.47	1.34

Feature Scaling (Examples)

Compute the Mean μ

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Calculation:

$$\mu = \frac{90 + 120 + 150 + 200}{4}$$

$$\mu = \frac{560}{4} = 140$$

✅ **Mean (μ) = 140**

Compute the Standard Deviation σ

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (x_i - \mu)^2}$$

Step 1.: Subtract the Mean from Each Value

Value x_i	$x_i - \mu$
90	$90 - 140 = -50$
120	$120 - 140 = -20$
150	$150 - 140 = 10$
200	$200 - 140 = 60$

Step 2: Square Each Difference

Difference	Squared
-50	2500
-20	400
10	100
60	3600

Step 3: Compute the Variance

Variance

$$= \frac{2500 + 400 + 100 + 3600}{4}$$

$$= \frac{6600}{4} = 1650$$

Step 4: Take Square Root

$$\sigma = \sqrt{1650} \approx 40.62$$

✅ **Standard Deviation (σ) ≈ 40.62**

Examples

3. Normalization

This method is more or less the same as the previous method but here instead of the minimum value we subtract each entry by the mean value of the whole data and then divide the results by the difference between the minimum and the maximum value.

$$X_{\text{scaled}} = \frac{X_i - X\text{mean}}{X_{\max} - X_{\min}}$$

Size	Room#
90	3
120	5
150	4
200	6

→ ?

Scaling for Different ML Models

✓ Standardization
✓ Robust Scaling (if outliers)

✓ Standardization
✓ Min-Max Scaling

Rely on distance calculations

✓ Min-Max Scaling
✓ Standardization
✓ Pixel scaling (/255)

Improves gradient stability

✗ No scaling required

Tree-based models split on thresholds

1. Does model rely on distances?
→ Yes → Apply scaling

2. Are there strong outliers?
→ Yes → Use Robust Scaling

3. Is data highly skewed?
→ Yes → Log/Power Transform

4. Is model tree-based?
→ Usually no scaling needed

Decision Framework for Choosing Scaling Methods

Does model rely on distances?	Yes → Apply Standardization or Min-Max
Are there strong outliers?	Yes → Use Robust Scaling
Is data highly skewed?	Yes → Use Log/Power Transform
Is model tree-based?	Usually no scaling needed

Critical: Avoid Data Leakage

- Always compute scaling parameters on training set only
- Apply same parameters to test set

Failure to do so causes:

Data leakage leads to overly optimistic evaluation

Final Insight: Feature Scaling

not optional preprocessing

It directly affects:

Optimization Speed
Faster convergence

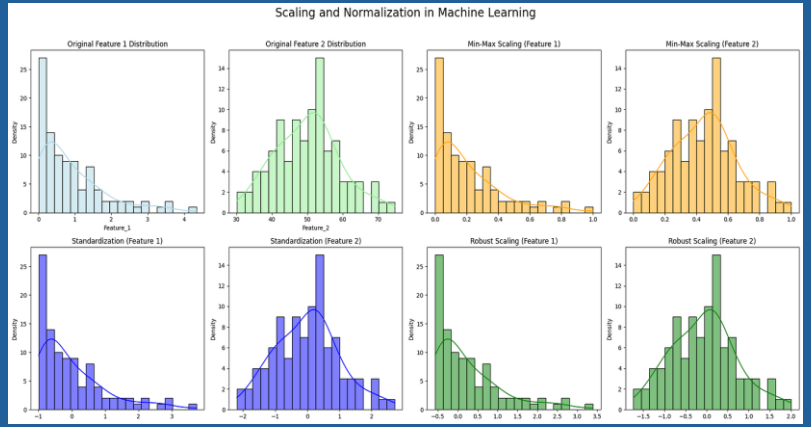
Model Stability
Numerical precision

Performance
Predictive accuracy

Robust Systems
High-performing ML

- Understand when to apply each technique
- Avoid common pitfalls like data leakage
- Build production-ready ML systems

Feature Scaling



Why Scaling Matters

- Different scales → slow convergence
- Similar scales → faster gradient descent
- Prevents one feature from dominating

Mean Norm.
$(x - \mu) / (\max - \min)$
Range: ~[-1, 1]

Standard.
$(x - \mu) / \sigma$
$\mu=0, \sigma=1$

Min-Max
$(x - \min) / (\max - \min)$
Range: [0, 1]

Problem: House features on different scales

Before Scaling	After Scaling
Size: 0 - 2000 sq ft	Size: 0.0 - 1.0
Bedrooms: 1 - 5	Bedrooms: 0.0 - 1.0

Result: Converges 10× faster ✓

Target Range

Aim for all features in `[-1, 1]` or `[0, 1]`

Feature Engineering: Scaling, Normalization, and Standardization

https://colab.research.google.com/drive/1wfXZbISv_bJtM05NXTWUGP794X0WzWk5?usp=sharing

Feature Engineering Introduction

Feature Engineering Introduction

- Creating **new features**
- Transforming **existing features**
- Selecting **useful features**

Key Insight

Feature engineering has a **larger impact** than the choice of algorithm

Even a simple algorithm can perform **extremely well** if features are well designed

Input variable used by machine learning model

Features	Target
• Size • Rooms • Location • Age	**Price**

Data Cleaning → Transformation → Creation

Encoding → Selection → Scaling

Feature Transformation

What is Feature Transformation?

Changes the **mathematical representation** of variables

Square Root Transformation

$$x' = \sqrt{x}$$

To **reduce variance**

Log Transformation

$$x' = \log(x)$$

- For **skewed data**
- Example: Income distribution

Polynomial Transformation

$$x^2 \quad x^3 \quad \ldots$$

Create **new features** for polynomial regression

Feature Scaling & Encoding

Normalization (Min-Max)

$$x' = (x - \min) / (\max - \min)$$

Range: [0, 1]

Standardization (Z-Score)

$$x' = (x - \mu) / \sigma$$

μ = mean, σ = standard deviation

Used in: **SVM, Neural Networks, KNN**

One-Hot Encoding

Categorical → Binary columns

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0

Label Encoding

Assigns numeric codes to categories

Red → 0

Blue → 1

May introduce **false ordering**

Feature Creation & Selection

Generate new features from existing ones

Example 1: BMI Calculation

Original Features
Height, Weight →

New Feature
BMI
$= W / H^2$

Example 2: Area Calculation

Original Features
Length, Width →

New Feature
Area
$= L \times W$

Remove irrelevant features

- Add noise
- Increase time
- Reduce performance

Filter Methods
Correlation, Chi-square, Mutual information

Wrapper Methods
Forward Selection, Backward Elimination

Embedded Methods
LASSO, Decision Trees, Random Forest

Feature Engineering Example & Importance

Original Features

- Size
- Age
- Rooms
- Location

Engineered Features

- Price/m²
- Distance to center
- Age²
- Rooms per area

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
# Feature creation
df["size_per_room"] = df["size"] / df["rooms"]
# Scaling
scaler = StandardScaler()
scaled = scaler.fit_transform(df)
```

- Learn patterns easily
- Reduce noise
- Improve accuracy
- Reduce overfitting

Many ML competitions are won by **better feature engineering**

Spam Detection	Fraud Detection	Recommendations
Word frequency, links	Transaction patterns	User history

Lab

https://www.kaggle.com/code/seneralkan/advanced-feature-engineering

Machine Learning Fundamentals

Overfitting, Underfitting

Understanding Model Performance and Parameter Estimation

Overfitting **Underfitting** **Normal Equation**

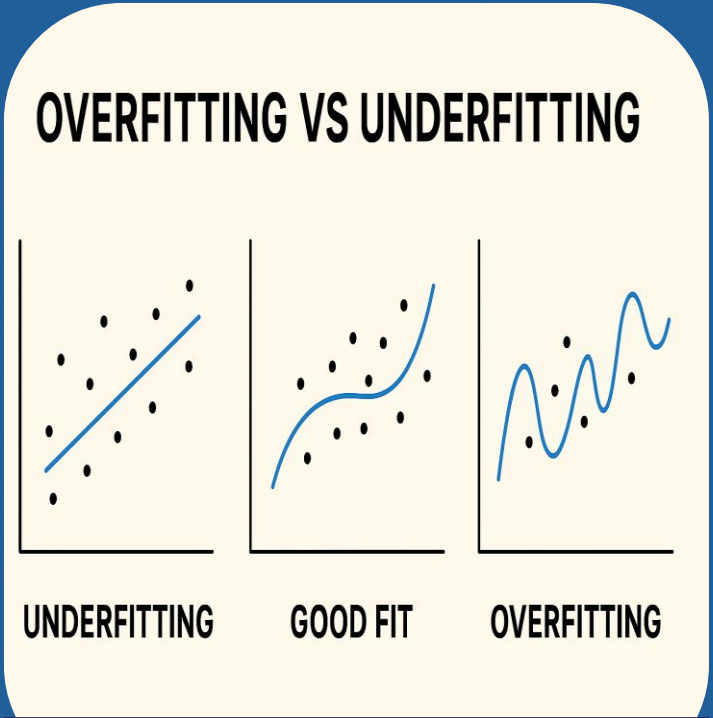
Overfitting and Underfitting

Two Common Problems

When training a machine learning model, problems can occur that affect the model's ability to **generalize to unseen data**

Overfitting
Model learns training data too well, including noise

Underfitting
Model is too simple to capture underlying patterns



Model Fitting Comparison

Overfitting

Model learns training data **too well**, including noise and fluctuations that don't represent the true underlying pattern

Training Performance

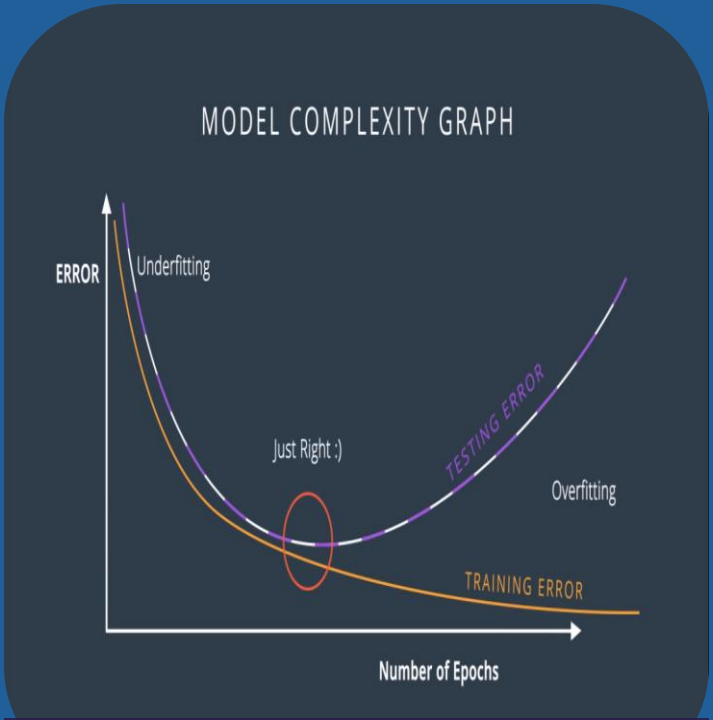
Very high accuracy on training dataset

Testing Performance

Poor performance on new or testing data

Learning Behavior

Memorizes training examples instead of learning general patterns



Model Complexity vs Error

Causes of Overfitting

Excessively Complex Models

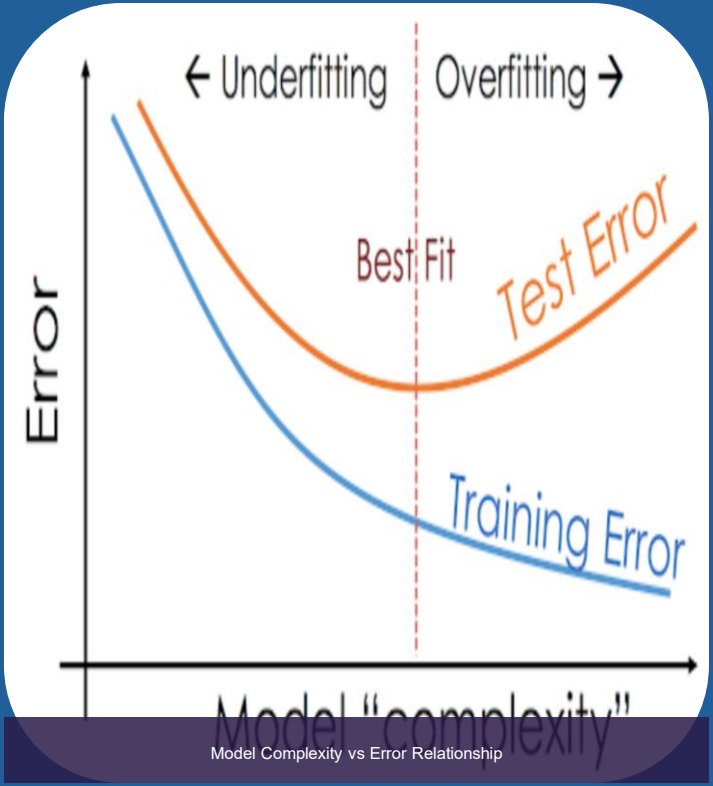
Models with too many parameters relative to training data complexity

Too Many Features

High dimensionality with insufficient samples to support feature count

Insufficient Training Data

Limited samples prevent model from learning generalizable patterns



Model Complexity vs Error Relationship

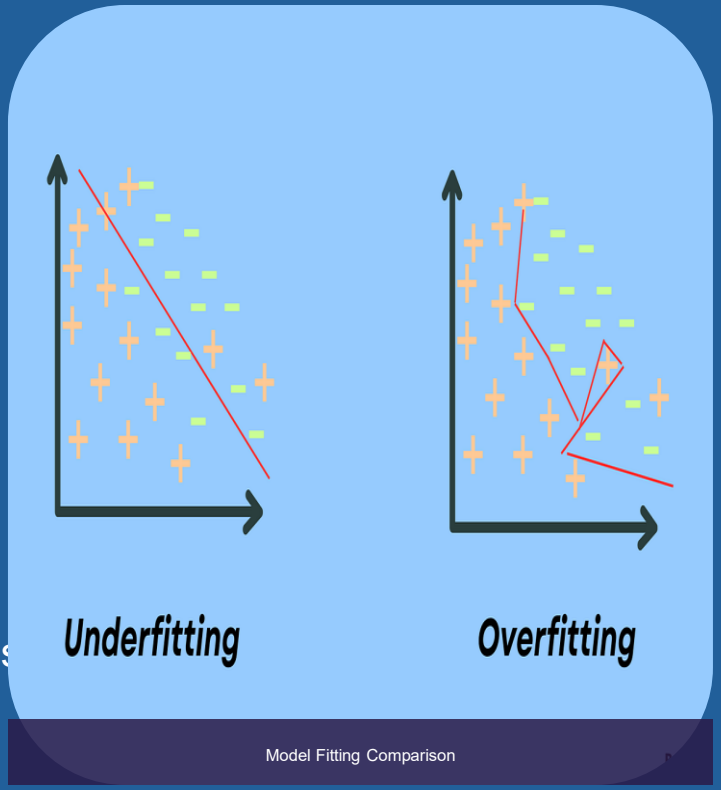
Underfitting

Model is **too simple** to capture the underlying structure of the data

Training Performance
Poor performance on training data

Testing Performance
Poor performance on testing data

Key Insight
Fails to capture essential patterns in both datas



Causes of Underfitting

Oversimplified Models

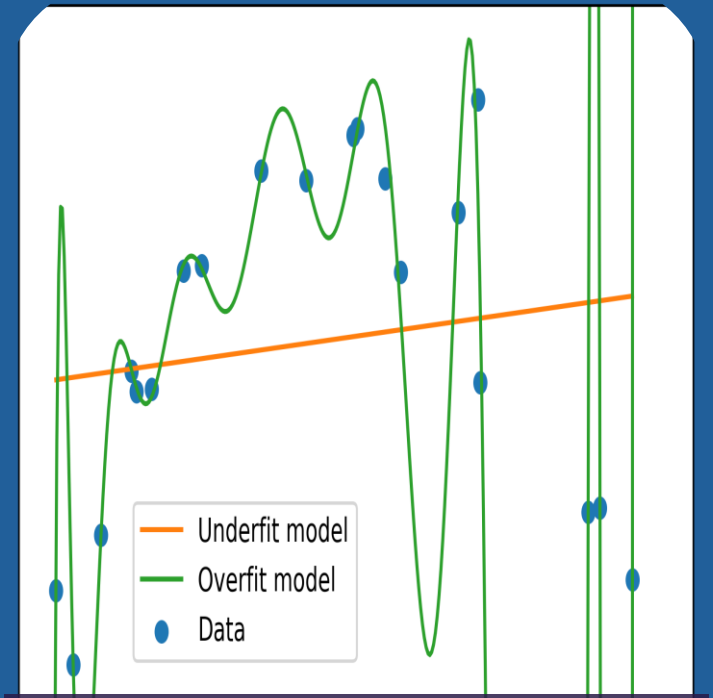
Models too simple to capture data complexity

Insufficient Features

Poorly designed or inadequate feature representation

Inadequate Training

Insufficient training time or iterations



Model Comparison: Underfitting vs Overfitting

Model Generalization & Bias-Variance Tradeoff

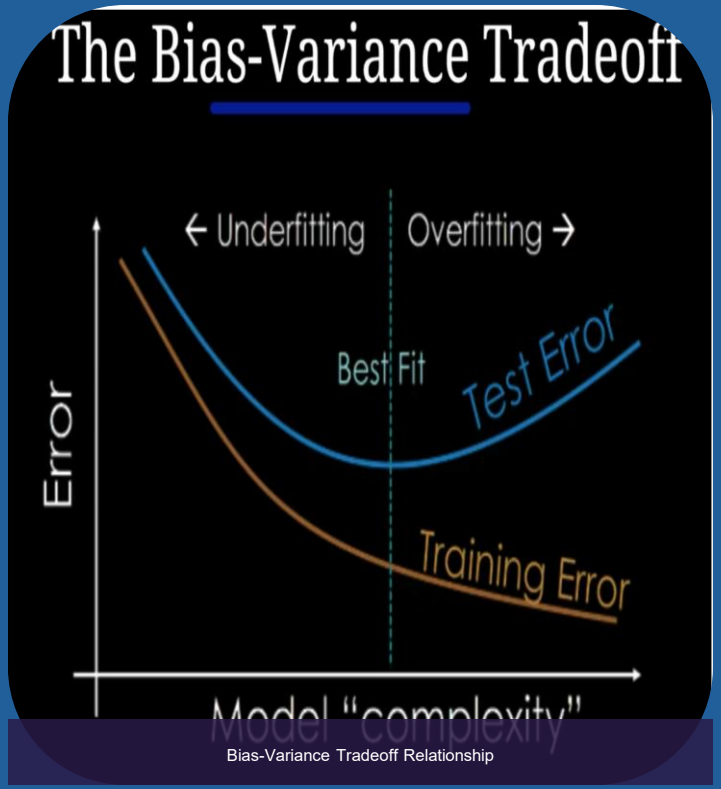
The Challenge

Achieve **balance** between model complexity and generalization ability

Bias
Error due to wrong assumptions

Variance
Sensitivity to small changes in the training data

Optimal Balance
Maintains good performance on **both training and unseen data**



Bias-Variance Tradeoff Relationship

Model Behavior: Bias–Variance Tradeoff

To understand how a model generalizes to unseen data, we must understand the Bias-Variance tradeoff. The total expected error of a model can be decomposed into three parts:

$$\textbf{Error} = \text{bias}^2 + \text{Variance} + \text{Irreducible Error}.$$

The goal of machine learning is to find the optimal sweet spot—minimizing both bias and variance simultaneously.

Model Behavior: Bias–Variance Tradeoff

- **Bias** is the error introduced by approximating a real-world problem, which may be complex, by a much simpler model.

Bias is the error that happens when a model is **too simple** to understand the real pattern in the data.

👉 In other words: The model **makes strong assumptions** that are not true.

🎯 **Key Idea**

■ High Bias = Model is too simple ■ It **misses important patterns** ■ Leads to **underfitting**

⚠️ **Signs of High Bias**

- Model performs **badly on training data**
- Model performs **badly on test data**
- Predictions are **too simple**

Model Behavior: Bias–Variance Tradeoff

📊 Bias Examples

Example 1: Straight Line vs Curved Data
Imagine the real data looks like a curve:
•Real relationship: curved (nonlinear)
•Model used: straight line
👉 The model cannot follow the curve → large error
✓ This is **high bias**

Example 2: Exam Scores
Suppose:
•Students who study more usually get higher grades
•But the relationship is not perfectly linear
If the model says:
“Grade = 2 × study_hours” (always a straight line)
👉 This is too simple
👉 It ignores real variations
✓ Result: high bias

Example 3: Image Classification
•Real task: recognize cats vs dogs
•Model only checks: “Does it have 4 legs?”
👉 Many animals have 4 legs
👉 Model ignores important features (ears, face, etc.)
✓ This is high bias

Model Behavior: Bias–Variance Tradeoff

Variance measures how much a model's predictions **change when we use different training data**.

👉 In simple words:

The model is **too sensitive to the data it sees**.

🧠 Intuition

Think of variance as:

"The model is *too sensitive* and reacts to every small detail."

It learns not only the real pattern, but also the **noise (random errors)**.

🎯 Key Idea

- High Variance = Model is too complex
- It **memorizes the training data**
- Leads to **overfitting**

⚠️ Signs of High Variance

- Very **low error on training data**
- **High error on test data**
- Model changes a lot with new data

Model Behavior: Bias–Variance Tradeoff

📊 Variance Examples

Example 1: Curve Fitting

Imagine:

•Real pattern: smooth curve

•Model: very complex curve that goes through every point

👉 It fits **every small fluctuation**

👉 If we change the dataset slightly, the curve changes a lot

✓ This is **high variance**

Example 2: Exam Scores

•Training data: small group of students

•Model learns very detailed patterns (even random behavior)

Now if we train on a **different group**:

👉 The model gives **very different predictions**

✓ This shows high variance

Example 3: Memorizing Instead of Learning

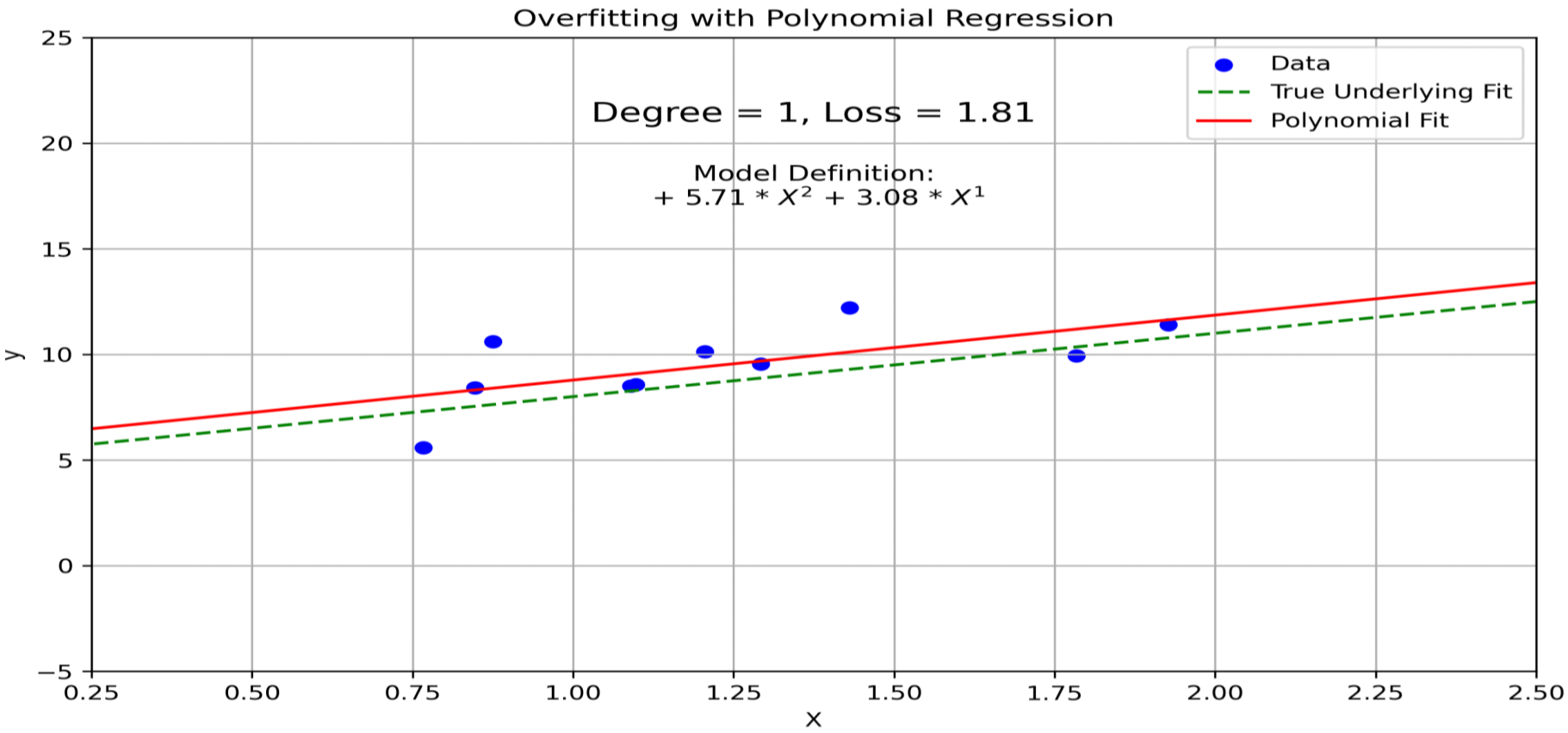
•Model remembers exact answers for training data

•But when new data comes → it fails

👉 Because it learned **details, not the general rule**

✓ High variance

Model Behavior: Bias–Variance Tradeoff



Techniques to Handle Overfitting

When you detect overfitting, you can employ several strategies:

1. **Cross-Validation:** Use techniques like k-fold cross-validation to ensure the model evaluates well across different subsets of data.
2. **More Data / Data Augmentation:** Providing more data forces the model to generalize. If getting new data is impossible, we can synthetically augment existing data (especially common in image processing).
3. **Feature Selection:** Drop irrelevant or noisy features that the model might be memorizing.
4. **Early Stopping:** In iterative algorithms (like Neural Networks), halt training when validation error begins to rise, even if training error is still falling.
5. **Model Simplification:** Decrease the degree of a polynomial, or decrease the depth of a decision tree.

Techniques to Handle Overfitting

	Overfitting	Undrfitting
Data set	Increase Training Set	Increase Training Set
	Reduce features	Add features
	Validation: • Cross validation	Validation: • Cross validation
Model	Decrease model complexity	Increase model complexity
	Regularization (Increase)	Regularization (Decrease)
	Ensemble (Bagging, Boosting)	Ensemble (Bagging, Boosting)
	Early stopping	Early stopping

Regularization

Machine Learning

Regularization

📌 Definition

Regularization adds a **penalty term** to the loss function to control model complexity.

- Forces the model to be **simpler**
- Better **generalization** to new data

Method	Effect
L1	Drives some weights to zero
L2	Shrinks weights smoothly

Feature	L1	L2
Feature selection	Yes	No
Stability	Less stable	More stable
Use case	Sparse data	General

Regularization Term

Mathematical Formulations

L2 Regularization (Ridge)

$$J(\theta) = \boxed{\sum (y - \hat{y})^2} + \lambda \sum \theta^2$$

MSE

L1 Regularization (Lasso)

$$J(\theta) = \boxed{\sum (y - \hat{y})^2} + \lambda \sum |\theta|$$

⚠ Overfitting Problem

- Fits training data very well
- Poor performance on new data

💡 Key Insight

A good model performs well on **unseen data**, not just training data

Intuition Behind Regularization

× Without Regularization

- Fits every data point
- Complex, wiggly curve
- **Overfitting**

✓ With Regularization

- Smoother model
- Ignores noise
- **Better generalization**

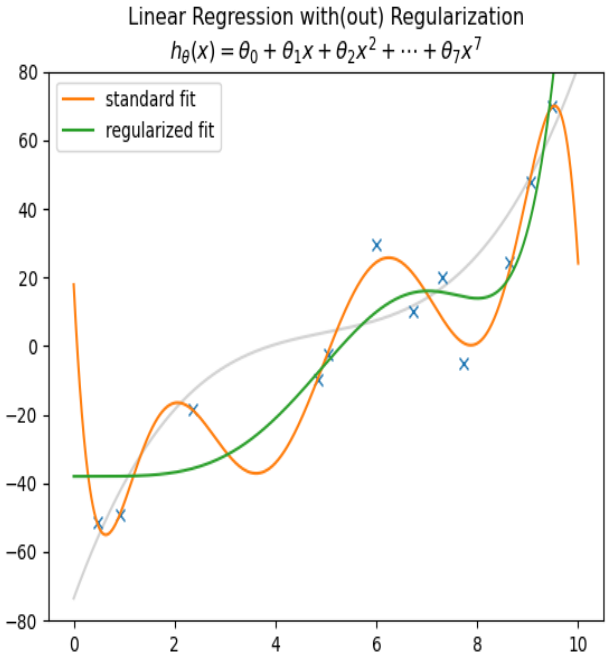
Key Insight

Small parameters = Simple model = Better generalization

Regularization prevents parameters from becoming too large, resulting in a **simpler, more generalizable model**

Effect of Lambda (λ)

Effect of Lambda (λ) Regularization Parameter



λ Value Impact

× λ = 0:

No regularization

overfitting

Small λ:

Slight simplification

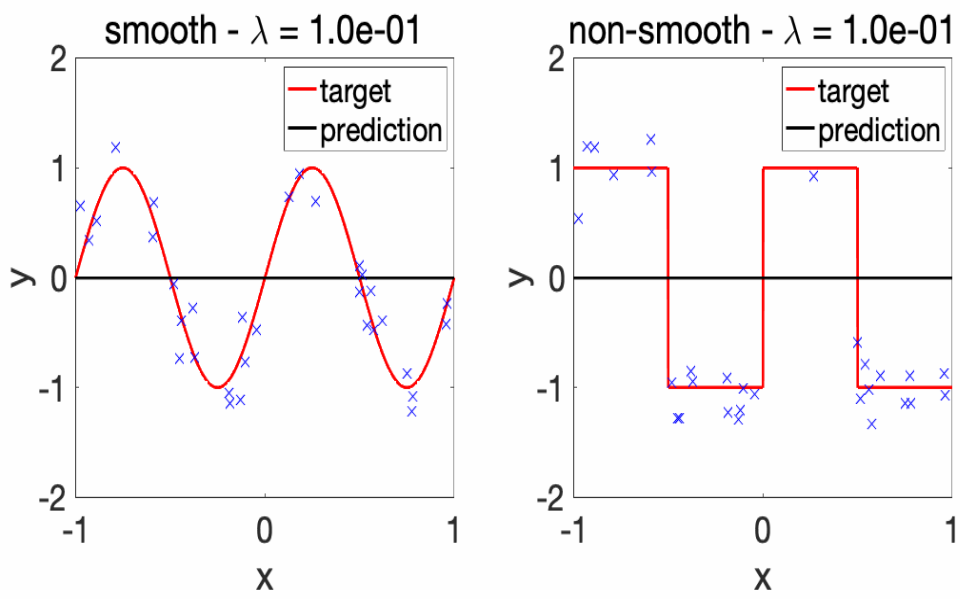
Large λ:
Very simple model

Very large λ:

Underfitting

- Extremely large λ → Parameters near zero →
- Model ignores input features

How does regularization work?



source: Image

Regularization works by adding a penalty to the model's parameters, which helps prevent overfitting and improves generalization.

When the regularization parameter (λ) is properly adjusted, the model becomes smoother and less sensitive to noise. **A higher λ** increases the penalty, simplifying the model, while a **lower λ** allows more complexity.

Overall, regularization controls model complexity by penalizing large parameter values during training.

Summary

Technique	Advantages	Limitations
Feature Engineering	Boosts accuracy	Time-intensive
Scaling	Improves convergence	Not always needed
Regularization	Prevents overfitting	Requires tuning